UNIVERSIDAD NACIONAL DE TRUJILLO

FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS



INFORME ACADÉMICO

CONECTAR UNA BASE DE DATOS MYSQL EN JAVA

Programación Orientada a Objetos II

Docente:

Zoraida Yanet Vidal Melgarejo

Integrantes:

Tejada Chiclayo José Angel

Vásquez Juape José Andrés

Chávez Namoc Leonardo Gabriel

Sánchez Santos Miguel Antonio

Ruiz Terrones Emily Ayleen

Guadalupe - Perú

2026

Índice

Índice ii
Resumen iv
Introducción 1
Objetivos 2
Objetivo general 2
Objetivos específicos 2
Marco teórico 2
MySQL: definición 2
Principales características de MySQL 3
JDBC en Java 3
MySQL Connector/J 4
Metodología 4
Desarrollo técnico 5
Creación de una base de datos en MySQL 5
Creación de una tabla de ejemplo 6
Datos de prueba 6
Pasos para conectar MySQL con Java 6
Ejemplo de clase de conexión 7
Ejemplo de consulta con PreparedStatement 7
Relación con la programación orientada a objetos 8
Resultados 9
Discusión 9
Errores comunes y soluciones 10
Buenas prácticas 10
Conclusiones 11
Recomendaciones 12
Referencias 13

Resumen

El presente informe analiza el proceso para conectar una base de datos MySQL con una aplicación desarrollada en Java, considerando el contexto académico del curso Programación Orientada a Objetos II. El problema abordado consiste en comprender cómo una aplicación puede dejar de trabajar solo con datos temporales en memoria y pasar a conservar información de manera persistente mediante un sistema gestor de bases de datos. Para ello, se empleó un enfoque descriptivo y aplicado, basado en la revisión de documentación técnica oficial y en la construcción de un ejemplo básico con MySQL Workbench, NetBeans, JDBC y MySQL Connector/J. Los principales hallazgos muestran que MySQL permite organizar información en tablas relacionales, que la creación de la base requiere sentencias SQL como CREATE DATABASE y CREATE TABLE, y que Java necesita una URL JDBC, credenciales válidas y el controlador adecuado para establecer la conexión. Asimismo, se evidencia que encapsular la conexión en una clase mejora la organización del proyecto y se relaciona directamente con los principios de la programación orientada a objetos. En Ingeniería de Sistemas, este aprendizaje fortalece la construcción de aplicaciones más mantenibles, escalables y cercanas a escenarios reales de desarrollo.

Palabras clave: *MySQL, Java, JDBC, NetBeans, MySQL Workbench.*

Introducción

En el desarrollo de software actual, las aplicaciones rara vez trabajan únicamente con datos temporales almacenados en memoria. Por el contrario, la mayoría de sistemas informáticos necesita registrar, consultar, modificar y proteger información de manera persistente. En ese contexto, la conexión entre un lenguaje de programación y un sistema gestor de bases de datos se convierte en una competencia esencial para el estudiante de Programación Orientada a Objetos II, ya que permite construir aplicaciones más completas, organizadas y cercanas a escenarios reales.

Java es un lenguaje ampliamente utilizado para la creación de aplicaciones de escritorio, empresariales y académicas. Para comunicarse con bases de datos relacionales, Java utiliza JDBC, una API que permite establecer conexiones, ejecutar sentencias SQL, procesar resultados y manejar errores de base de datos. Oracle (2024) presenta JDBC como el conjunto de recursos que permite establecer conexiones, procesar sentencias SQL, trabajar con consultas preparadas y manejar transacciones.

MySQL, por su parte, es uno de los sistemas gestores de bases de datos relacionales más utilizados en entornos educativos y profesionales. Su integración con Java se realiza mediante MySQL Connector/J, controlador JDBC oficial que permite que una aplicación Java se comunique con un servidor MySQL. En el entorno del curso, MySQL Workbench facilita la creación y prueba de la base de datos, mientras que NetBeans permite organizar el proyecto Java, agregar librerías y ejecutar las clases de conexión.

La necesidad técnica que orienta este informe es explicar cómo preparar una base de datos MySQL y cómo establecer una conexión desde Java sin perder de vista buenas prácticas básicas de programación orientada a objetos. El alcance se limita a una conexión local, una tabla de ejemplo y una consulta SELECT, por lo que no se desarrolla un sistema completo ni un CRUD avanzado. Esta delimitación permite mantener el análisis acorde con el nivel de tercer ciclo y con los objetivos del curso.

El documento se organiza en objetivos, marco teórico, metodología, desarrollo técnico, resultados, discusión, errores comunes, buenas prácticas, conclusiones, recomendaciones y referencias. De esta manera, el informe combina fundamentos conceptuales con un ejemplo práctico aplicable en NetBeans y MySQL Workbench.

Objetivos

Objetivo general

Explicar el proceso de creación de una base de datos MySQL y su conexión desde una aplicación desarrollada en Java mediante JDBC, considerando fundamentos teóricos, pasos técnicos y buenas prácticas de programación orientada a objetos.

Objetivos específicos

1. Definir MySQL y describir sus principales características como sistema gestor de bases de datos relacional.
2. Describir los pasos necesarios para crear una base de datos y una tabla de ejemplo en MySQL.
3. Explicar el rol de JDBC y MySQL Connector/J dentro de la conexión entre Java y MySQL.
4. Presentar un ejemplo básico de conexión desde Java utilizando una clase reutilizable y una consulta con PreparedStatement.
5. Identificar buenas prácticas y errores frecuentes al conectar aplicaciones Java con bases de datos MySQL.

Marco teórico

MySQL: definición

MySQL es un sistema gestor de bases de datos relacional que permite almacenar información organizada en tablas, establecer relaciones entre datos y realizar operaciones mediante el lenguaje SQL. En una base de datos relacional, la información se representa en estructuras formadas por filas y columnas, lo que facilita organizar entidades como estudiantes, cursos, usuarios, productos o transacciones.

La documentación oficial de MySQL 8.4 indica que este sistema administra bases de datos mediante objetos como esquemas, tablas, usuarios y privilegios, y que el servidor debe encargarse de gestionar los archivos internos de la base de datos (Oracle, 2026d). Este punto es importante porque en un trabajo académico conviene apoyarse en fuentes oficiales y no en tutoriales informales que pueden estar desactualizados.

En términos prácticos, MySQL cumple la función de administrar datos de manera persistente. Esto significa que los datos no desaparecen al cerrar la aplicación, sino que permanecen guardados en el servidor de base de datos. Para una aplicación Java, esta

capacidad resulta fundamental cuando se requiere registrar información de estudiantes, usuarios, ventas, inventarios o cualquier otro conjunto de datos que deba conservarse.

Principales características de MySQL

Entre las características más relevantes de MySQL se encuentra su modelo relacional. Este modelo permite organizar datos en tablas y vincularlas mediante claves primarias y foráneas. Gracias a ello, una aplicación puede mantener información estructurada y reducir la duplicidad de datos. Por ejemplo, en un sistema académico una tabla de estudiantes podría relacionarse con una tabla de matrículas o cursos.

Otra característica importante es el uso de SQL, lenguaje estándar para definir, consultar y manipular datos. Con SQL es posible crear bases de datos, crear tablas, insertar registros, consultar información, actualizar datos y eliminar registros. Esta uniformidad facilita que los estudiantes comprendan los fundamentos de gestión de información y puedan aplicarlos en distintos sistemas gestores de bases de datos.

MySQL también incorpora administración de usuarios y privilegios. Esta característica permite controlar qué usuarios pueden conectarse al servidor y qué operaciones están autorizados a realizar. En un entorno de desarrollo académico se suele trabajar con usuarios locales; sin embargo, en sistemas reales es indispensable limitar permisos para proteger la información.

Asimismo, MySQL es compatible con distintos sistemas operativos y puede integrarse con varios lenguajes de programación mediante conectores. En el caso de Java, la integración se realiza mediante MySQL Connector/J, que actúa como puente entre la API JDBC y el servidor MySQL. Esta integración permite que el programa Java ejecute instrucciones SQL sin implementar directamente el protocolo interno de comunicación con la base de datos.

JDBC en Java

JDBC, sigla de Java Database Connectivity, es la API de Java utilizada para acceder a bases de datos relacionales. Su importancia radica en que proporciona clases e interfaces estandarizadas para establecer conexiones, enviar sentencias SQL, recibir resultados y controlar excepciones. Oracle (2024) organiza los contenidos básicos de JDBC alrededor de acciones como establecer una conexión, procesar sentencias SQL, manejar SQLException, recuperar datos mediante ResultSet y utilizar PreparedStatement.

Desde la perspectiva de Programación Orientada a Objetos II, JDBC también permite aplicar principios de diseño. La conexión con la base de datos puede

encapsularse en una clase especializada, lo que evita repetir código en varias partes del programa. Esta separación facilita el mantenimiento, mejora la legibilidad y permite que otras clases se concentren en la lógica del problema.

Un elemento clave de JDBC es la interfaz Connection, que representa una conexión activa con la base de datos. A partir de ella se pueden crear objetos PreparedStatement para ejecutar consultas parametrizadas. El uso de PreparedStatement es preferible frente a la concatenación directa de cadenas SQL, porque mejora la organización del código y ayuda a reducir riesgos asociados con entradas maliciosas o errores de sintaxis (Oracle, 2026f).

MySQL Connector/J

MySQL Connector/J es el controlador JDBC oficial para conectar aplicaciones Java con MySQL. Este controlador implementa la interfaz java.sql.Driver y permite que DriverManager identifique cómo abrir una conexión hacia un servidor MySQL. La documentación oficial indica que la clase del driver es com.mysql.cj.jdbc.Driver (Oracle, 2026b).

La conexión se define mediante una URL JDBC. Según la documentación de Connector/J, la sintaxis general de una URL de conexión se compone de protocolo, servidor, base de datos y propiedades adicionales. Para una conexión JDBC básica con MySQL se utiliza el protocolo jdbc:mysql: (Oracle, 2026a). Un ejemplo común en ambiente local es jdbc:mysql://localhost:3306/poo_java.

En NetBeans, el conector puede incorporarse agregando el archivo JAR a las librerías del proyecto o mediante una dependencia si el proyecto usa Maven. En ambos casos, el objetivo es que el programa pueda cargar el controlador y que DriverManager establezca la conexión solicitada (Oracle, 2026g).

Metodología

El informe emplea una investigación descriptiva y aplicada. Es descriptiva porque explica conceptos técnicos como MySQL, JDBC, Connector/J, URL de conexión y PreparedStatement; y es aplicada porque organiza esos conceptos en un procedimiento práctico para crear una base de datos y conectarla desde una aplicación Java.

El enfoque utilizado es cualitativo, debido a que se interpreta documentación técnica y se analiza su aplicación en un caso académico. Como técnica de recolección se utilizó la revisión bibliográfica de documentación oficial de Oracle, MySQL y Apache

NetBeans, complementada con un ejemplo práctico orientado al curso Programación Orientada a Objetos II.

Las fuentes se seleccionaron considerando tres criterios: relación directa con el tema, actualidad de la documentación y autoridad técnica de la organización responsable. Por ello, se priorizaron manuales oficiales, documentación de API y guías de herramientas usadas en el entorno de trabajo, como MySQL Workbench y NetBeans.

El procedimiento de análisis consistió en identificar primero los conceptos necesarios para entender la conexión, luego describir la creación de la base de datos en MySQL Workbench, después presentar los pasos de configuración del proyecto en NetBeans y finalmente interpretar los resultados desde una perspectiva de organización del código orientada a objetos.

Desarrollo técnico

Creación de una base de datos en MySQL

El primer paso técnico consiste en crear una base de datos donde se almacenará la información de la aplicación. MySQL permite crear una base mediante la sentencia CREATE DATABASE. La documentación oficial señala que esta sentencia crea una base de datos con el nombre indicado y requiere privilegio CREATE para ejecutarse (Oracle, 2026c). En un entorno académico, esta instrucción puede ejecutarse desde una pestaña SQL de MySQL Workbench.

La forma básica del comando es la siguiente:

```
CREATE DATABASE nombre_base_datos;
```

Para un proyecto académico de Programación Orientada a Objetos II, se puede usar una base llamada poo_java. Además, resulta recomendable especificar un conjunto de caracteres moderno como utf8mb4 y una colación compatible con texto en español. El comando sugerido es el siguiente:

```
CREATE DATABASE IF NOT EXISTS poo_java
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

En este comando, CREATE DATABASE ordena la creación de la base de datos; IF NOT EXISTS evita un error si la base ya existe; CHARACTER SET define el conjunto de caracteres usado por defecto; y COLLATE establece las reglas de comparación y ordenamiento de texto. Estas opciones ayudan a evitar problemas con tildes, eñes y otros caracteres comunes en español.

Creación de una tabla de ejemplo

Después de crear la base de datos, se debe seleccionar la base y crear una tabla. Para mantener el ejemplo simple y relacionado con un contexto académico, se propone una tabla llamada estudiante. Esta tabla contiene un identificador autoincremental, un nombre y un correo electrónico.

```
USE                                                                    poo_java;

CREATE                  TABLE                  estudiante                  (
    id          INT          AUTO_INCREMENT          PRIMARY          KEY,
    nombre               VARCHAR(80)              NOT              NULL,
    correo               VARCHAR(100)              NOT              NULL
);
```

La columna id funciona como clave primaria y permite identificar de manera única cada registro. La columna nombre almacena el nombre del estudiante y correo registra una dirección electrónica. La restricción NOT NULL indica que esos campos no deben quedar vacíos.

Datos de prueba

Para verificar la conexión desde Java, conviene insertar algunos datos iniciales. Estos datos permitirán ejecutar una consulta SELECT y comprobar que el programa recupera información desde MySQL.

```
INSERT      INTO      estudiante      (nombre,      correo)      VALUES
('Ana               Torres',              'ana.torres@unitru.edu.pe'),
('Luis             Ramírez',            'luis.ramirez@unitru.edu.pe'),
('María Sánchez', 'maria.sanchez@unitru.edu.pe');
```

Pasos para conectar MySQL con Java

Una vez creada la base de datos, el siguiente paso es preparar el proyecto Java. La conexión requiere que el proyecto tenga disponible MySQL Connector/J. En NetBeans, este paso puede realizarse desde las propiedades del proyecto agregando el JAR del conector a Libraries, o mediante Maven si el proyecto fue creado con esa configuración.

Los pasos generales para establecer la conexión son los siguientes:

- Crear o verificar la base de datos en MySQL Workbench.
- Agregar MySQL Connector/J al proyecto Java en NetBeans.
- Importar las clases JDBC necesarias: Connection, DriverManager, PreparedStatement, ResultSet y SQLException.
- Definir la URL de conexión, el usuario y la contraseña de MySQL.
- Abrir la conexión mediante DriverManager.getConnection.
- Preparar y ejecutar una consulta SQL con PreparedStatement.

- Leer los resultados con ResultSet.
- Cerrar los recursos mediante try-with-resources.
- Manejar posibles errores con SQLException.

Para un proyecto académico se recomienda usar una clase dedicada a la conexión. Esta decisión refleja un principio de diseño orientado a objetos: separar responsabilidades. La clase de conexión se encarga únicamente de abrir conexiones, mientras que otras clases pueden encargarse de consultar, insertar o procesar información.

Ejemplo de clase de conexión

La siguiente clase define los datos mínimos para conectarse a la base de datos poo_java. En un proyecto real, la contraseña no debería escribirse directamente en el código; sin embargo, para fines académicos se muestra como un valor de ejemplo que debe reemplazarse por la contraseña local del estudiante.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {
    private static final String URL =
"jdbc:mysql://localhost:3306/poo_java";
    private static final String USUARIO = "root";
    private static final String CLAVE = "tu_clave";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CLAVE);
    }
}
```

La constante URL indica el protocolo JDBC, el servidor local, el puerto 3306 y el nombre de la base de datos. Las constantes USUARIO y CLAVE representan las credenciales de acceso. El método getConnection devuelve un objeto Connection, que será utilizado por otras clases para ejecutar operaciones SQL.

Ejemplo de consulta con PreparedStatement

El siguiente ejemplo consulta los estudiantes registrados en la tabla estudiante. Se utiliza try-with-resources para cerrar automáticamente la conexión, el PreparedStatement y el ResultSet al finalizar la ejecución.

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ListarEstudiantes {
    public static void main(String[] args) {
        String sql = "SELECT id, nombre, correo FROM estudiante";
```

```
        try (Connection conn = ConexionBD.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                int id = rs.getInt("id");
                String nombre = rs.getString("nombre");
                String correo = rs.getString("correo");
                System.out.println(id + " - " + nombre + " - " +
correo);
            }
        } catch (SQLException e) {
            System.out.println("Error al consultar estudiantes: " +
e.getMessage());
        }
    }
}
```

Este código demuestra una conexión básica y suficiente para el objetivo del informe. Primero obtiene una conexión mediante ConexionBD.getConnection; luego prepara la consulta SELECT; después ejecuta la consulta; finalmente recorre los resultados e imprime cada registro. La misma estructura puede ampliarse para insertar, actualizar o eliminar datos, pero no se desarrolla un CRUD completo para mantener el informe centrado en la conexión.

Relación con la programación orientada a objetos

Aunque la conexión a una base de datos parece un procedimiento técnico, también puede analizarse desde los principios de la programación orientada a objetos. La clase ConexionBD encapsula la configuración necesaria para abrir una conexión. Esto evita que el resto del programa repita la URL, el usuario y la contraseña en varias clases.

Además, separar la conexión de la consulta permite construir posteriormente clases DAO, repositorios o servicios. Por ejemplo, una clase EstudianteDAO podría encargarse de listar, buscar o registrar estudiantes, mientras ConexionBD se mantiene como componente de infraestructura. Esta organización mejora la mantenibilidad del software y facilita pruebas o cambios futuros.

En un proyecto de NetBeans, esta separación también ayuda a ordenar paquetes. Una estructura sencilla podría ubicar ConexionBD en un paquete de conexión o infraestructura, las clases de entidad en un paquete modelo y las operaciones de consulta en una clase DAO. De ese modo, la conexión no queda mezclada con menús, ventanas o lógica de presentación.

Resultados

A partir del análisis realizado, se obtuvieron hallazgos directamente relacionados con los objetivos del informe. Los resultados se presentan de forma sintética para mostrar qué se logró comprender y aplicar en el proceso de conexión entre MySQL y Java.

Aspecto analizado	Resultado obtenido
MySQL	Se identificó como un gestor relacional adecuado para almacenar datos persistentes en proyectos académicos y aplicaciones Java.
Creación de base de datos	Se determinó que la base poo_java puede prepararse con CREATE DATABASE, USE, CREATE TABLE e INSERT desde MySQL Workbench.
Conexión desde Java	Se estableció que Java requiere JDBC, MySQL Connector/J, una URL de conexión, credenciales y manejo de SQLException.
POO II	Se confirmó que encapsular la conexión en una clase separada favorece la reutilización y la separación de responsabilidades.
Buenas prácticas	Se reconoció la importancia de PreparedStatement, try-with-resources, nombres descriptivos y protección de contraseñas reales.

Nota. La tabla resume los hallazgos del análisis técnico aplicado al contexto de Programación Orientada a Objetos II.

Discusión

Los resultados confirman que la conexión entre Java y MySQL no debe entenderse solo como una secuencia de instrucciones, sino como una integración entre conceptos de base de datos, API de programación y diseño orientado a objetos. La teoría revisada sobre JDBC indica que Java ofrece una capa estándar para comunicarse con bases de datos; el ejemplo desarrollado muestra cómo esa teoría se materializa mediante DriverManager, Connection, PreparedStatement y ResultSet.

En comparación con una solución improvisada donde la URL, el usuario y la contraseña se repiten en cada clase, el uso de una clase ConexionBD representa una mejora de organización. Esta decisión es sencilla, pero coherente con el nivel de POO II,

porque introduce la idea de responsabilidad única sin exigir todavía patrones más avanzados como DAO completo, pool de conexiones o frameworks de persistencia.

La elección de MySQL Workbench y NetBeans también es pertinente para un entorno académico. MySQL Workbench permite verificar visualmente la base de datos, ejecutar sentencias SQL y comprobar datos antes de probar el código Java; NetBeans facilita la ejecución del proyecto y la administración de librerías. Esta combinación reduce la dificultad inicial y permite concentrarse en comprender el proceso de conexión.

La principal limitación del informe es que el ejemplo trabaja con una conexión local, una tabla simple y una consulta de lectura. Por ello, no cubre autenticación avanzada, conexión remota, transacciones complejas, interfaz gráfica, CRUD completo ni despliegue en producción. Sin embargo, esta limitación es razonable para un informe introductorio de tercer ciclo, porque evita desviar el objetivo principal: comprender cómo Java se conecta a MySQL y cómo organizar esa conexión dentro del proyecto.

Errores comunes y soluciones

Durante la conexión entre Java y MySQL pueden aparecer errores frecuentes. Uno de ellos es No suitable driver found, que suele indicar que MySQL Connector/J no fue agregado correctamente al proyecto. En NetBeans, esto se corrige revisando que el archivo JAR se encuentre dentro de las librerías del proyecto o que la dependencia Maven esté correctamente configurada.

Otro error común es Access denied for user, relacionado con credenciales incorrectas o permisos insuficientes del usuario de MySQL. También puede presentarse Unknown database, cuando el nombre de la base de datos escrito en la URL no existe o no coincide exactamente con el nombre creado en MySQL Workbench.

Asimismo, Communications link failure suele aparecer cuando el servidor MySQL no está iniciado, el puerto no es correcto o existe un problema de conexión local. La solución requiere revisar paso a paso la configuración: verificar que MySQL esté activo, confirmar el nombre de la base de datos, comprobar usuario y contraseña, validar que el conector JDBC esté agregado al proyecto y asegurarse de que la URL siga el formato correcto.

Buenas prácticas

Al conectar una aplicación Java con MySQL, no basta con lograr que el programa funcione. También es necesario aplicar buenas prácticas que permitan escribir código seguro, legible y mantenible. Una práctica fundamental es no escribir contraseñas reales

en documentos académicos ni en repositorios públicos. En el informe se debe usar una contraseña de ejemplo y aclarar que debe reemplazarse localmente.

Otra práctica recomendable es usar PreparedStatement. Aunque el ejemplo presentado no recibe datos del usuario, esta clase es importante porque permite trabajar con parámetros y ayuda a evitar errores derivados de concatenar texto directamente en sentencias SQL. También se recomienda cerrar conexiones y recursos, preferiblemente con try-with-resources, ya que esta estructura reduce fugas de recursos y simplifica el manejo del cierre.

Además, se debe separar la lógica de conexión de la lógica de negocio. En un programa orientado a objetos, cada clase debe tener una responsabilidad clara. Por ello, una clase de conexión no debería imprimir menús ni procesar reglas del sistema; su función principal debe ser abrir conexiones a la base de datos.

Finalmente, conviene usar nombres descriptivos para bases de datos, tablas, columnas y clases. Nombres como poo_java, estudiante, ConexionBD y ListarEstudiantes permiten entender el propósito del código sin depender de comentarios excesivos. Esta claridad es especialmente valiosa en cursos de programación, donde el orden del proyecto también forma parte del aprendizaje.

Conclusiones

1. MySQL es un sistema gestor de bases de datos relacional que permite almacenar información estructurada y persistente. Su uso en aplicaciones Java resulta importante porque permite que los programas conserven datos más allá de la ejecución temporal en memoria.

2. Crear una base de datos en MySQL requiere comprender sentencias básicas como CREATE DATABASE, USE, CREATE TABLE e INSERT. Estos comandos forman la base del trabajo con datos y permiten preparar el entorno antes de conectar la aplicación Java.

3. La conexión entre Java y MySQL se realiza mediante JDBC y MySQL Connector/J. JDBC proporciona las clases e interfaces necesarias para abrir conexiones, ejecutar sentencias SQL, recuperar resultados y manejar errores, mientras que Connector/J permite que esas operaciones se comuniquen específicamente con MySQL.

4. Desde la perspectiva de Programación Orientada a Objetos II, separar la conexión en una clase propia mejora la organización del proyecto. Esta práctica favorece la reutilización, reduce duplicidad y prepara el camino para diseños más completos, como clases DAO o repositorios.

5. Finalmente, una conexión correcta no depende solo de escribir una URL y credenciales. También requiere agregar el driver adecuado, manejar SQLException, cerrar recursos, usar PreparedStatement y proteger información sensible. Estas buenas prácticas fortalecen la calidad técnica del programa y reflejan una forma más profesional de desarrollar software.

Recomendaciones

- Antes de programar la conexión en Java, verificar en MySQL Workbench que la base de datos, las tablas y los datos de prueba existan correctamente.
- Agregar MySQL Connector/J desde una fuente oficial y comprobar que NetBeans lo reconozca dentro de las librerías o dependencias del proyecto.
- Mantener una clase de conexión separada y evitar repetir URL, usuario y contraseña en distintas partes del programa.
- Usar PreparedStatement y try-with-resources desde los primeros ejercicios, incluso cuando el ejemplo sea pequeño, para formar hábitos de programación más seguros.
- En trabajos posteriores, ampliar el ejemplo hacia operaciones CRUD, clases DAO, interfaz gráfica y validación de datos, manteniendo la separación de responsabilidades.

Referencias

Apache Software Foundation. (2026). Connecting to a MySQL Database. Apache NetBeans. Recuperado el 5 de junio de 2026, de https://netbeans.apache.org/tutorial/main/kb/docs/ide/mysql/

Oracle. (2024). Lesson: JDBC basics. The Java Tutorials. Recuperado el 5 de junio de 2026, de https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

Oracle. (2026a). Connector/J reference: Connection URL syntax. MySQL Connector/J Developer Guide. Recuperado el 5 de junio de 2026, de https://dev.mysql.com/doc/connector-j/en/connector-j-reference-jdbc-url-format.html

Oracle. (2026b). Connector/J reference: Driver/Datasource class name. MySQL Connector/J Developer Guide. Recuperado el 5 de junio de 2026, de https://dev.mysql.com/doc/connector-j/en/connector-j-reference-driver-name.html

Oracle. (2026c). CREATE DATABASE statement. MySQL 8.4 Reference Manual. Recuperado el 5 de junio de 2026, de https://dev.mysql.com/doc/refman/8.4/en/create-database.html

Oracle. (2026d). MySQL 8.4 reference manual. MySQL. Recuperado el 5 de junio de 2026, de https://dev.mysql.com/doc/refman/8.4/en/

Oracle. (2026e). MySQL Workbench Reference Manual. MySQL. Recuperado el 5 de junio de 2026, de https://dev.mysql.com/doc/workbench/en/

Oracle. (2026f). PreparedStatement. Java SE 26 & JDK 26 API Specification. Recuperado el 5 de junio de 2026, de https://docs.oracle.com/en/java/javase/26/docs/api/java.sql/java/sql/PreparedStatement.html

Oracle. (2026g). DriverManager. Java SE 24 & JDK 24 API Specification. Recuperado el 5 de junio de 2026, de https://docs.oracle.com/en/java/javase/24/docs/api/java.sql/java/sql/DriverManager.html